

Introduction

Booking systems look simple from the outside: pick a seat, pay, done. Underneath, most cinema and airline booking UIs don't let you pick *any* four adjacent seats — they restrict you to a small number of fixed blocks, because fixed blocks are dramatically cheaper to search and validate than arbitrary ranges. LeetCode 1386, Cinema Seat Allocation, is a distilled version of that real constraint, and it's a great interview problem because it rewards you for noticing the restriction rather than brute-forcing a general "four consecutive free seats" scan.

This problem shows up in interviews because it tests three separate skills at once: can you spot when a search space is smaller than it looks, can you represent state compactly (bitmasking), and can you reason about which parts of a grid actually matter versus which can be skipped entirely. It's also a natural bridge into system design — the same problem, extended slightly, is exactly what a real seat-locking service has to solve under concurrent bookings.

Problem Overview

You're given a cinema with `n` rows, each containing exactly ten seats, numbered 1 through 10. Some seats are already reserved, given to you as a list of `[row, seat]` pairs. Your job is to compute the maximum number of four-person groups that can be seated together, where "together" has a specific meaning: each group must occupy one of exactly three fixed four-seat blocks within a row:

- Block A: seats 2, 3, 4, 5
- Block B: seats 4, 5, 6, 7
- Block C: seats 6, 7, 8, 9

Seats 1 and 10 are never part of any block — they exist, but no group can ever sit there, and a reservation on either of them never affects anything.

You need to return the total number of groups across all rows.

Example

Take a small cinema with three rows and these reservations:

- Row 1: seats 2, 3, 8
- Row 2: seat 6

- Row 3: seats 1, 10

Row 1: Block A needs seat 2, which is reserved — Block A is dead. Block B needs seats 4–7, all open — that's one group. Block C needs seats 6–9, but seat 8 is reserved — Block C is dead. Row 1 contributes 1 group.

Row 2: Block A needs seats 2–5, all open — one group. Block B needs seat 6, reserved — dead. Block C needs seat 6, reserved — dead. Row 2 contributes 1 group.

Row 3: Seats 1 and 10 are reserved, but neither one touches any block. The row is effectively empty for our purposes, so it seats two full groups (Block A and Block C, which never overlap). Row 3 contributes 2 groups.

Total: $1 + 1 + 2 = 4$ **groups**, which matches the expected output for this classic case.

Intuition

The naive instinct is to scan every row left to right, looking for any run of four consecutive free seats. That's a reasonable algorithm for a *general* version of this problem, but it's solving a harder problem than the one being asked. Here, groups can only sit in three specific places, so instead of scanning ten seats for arbitrary runs, you only ever need to check three fixed positions per row.

The next trap is assuming a row can hold at most one group, because Block B overlaps both Block A and Block C on shared seats (4, 5 and 6, 7 respectively). That's true for B, but Block A and Block C themselves share nothing — they occupy seats 2–5 and 6–9 with no overlap. So a fully open row seats two groups, not one. The only reason a row is capped at one group is if a reservation happens to knock out enough blocks that only one non-overlapping option survives.

Once you see the row is really just "check three fixed windows, take the best non-conflicting combination," the natural representation is: for each row, only the seats between 2 and 9 matter — pack their reserved/free state into a single number, and each block becomes a fixed set of bits. Checking "is this block fully free" becomes checking whether a fixed bitmask ANDed with the row's reservation mask is zero.

Brute Force Solution

Idea: For every row that has at least one reservation, build a boolean array of the ten seats. For each of the three block starting points (2, 4, 6), check whether all four seats in that range are unreserved.

Algorithm: 1. Group reservations by row. 2. For each row with reservations, build a 10-element boolean array. 3. For each block (start at 2, 4, 6), check if all four seats in that range are free. 4. Track which blocks are free, then count non-overlapping combinations (A+C is always compatible, B is only usable alone). 5. Rows with no reservations get 2 groups automatically.

Advantages: Easy to reason about, no bit tricks required, trivial to verify by hand.

Disadvantages: Building and scanning a 10-element array per row is unnecessary overhead — you're paying array allocation and iteration cost for information that only ever depends on 8 bits.

Time complexity: $O(n)$ where n is the number of reservations, since only rows with reservations get processed, and each check is $O(1)$ work against a fixed-size array.

Space complexity: $O(k)$ where k is the number of distinct rows with reservations, for storing the boolean arrays.

The brute force approach is already close to optimal in big-O terms — the real win of the optimized version is a smaller constant factor and cleaner logic, which matters a lot in a live interview where clarity is being judged alongside correctness.

Optimal Solution

The optimization replaces the boolean array with a single integer bitmask per row, and the array scan with a bitwise AND.

Step 1 — Only track rows that matter. Use a dictionary keyed by row number. If a row never appears in the reservation list, it's fully empty and worth exactly 2 groups — no need to store or process it at all.

Step 2 — Build a mask per row. For each reservation at seat s in row r , if $2 \leq s \leq 9$, set bit $(s - 2)$ in that row's mask. Seats 1 and 10 are skipped entirely since they never affect any block.

Step 3 — Represent the blocks as constant masks.

Block Seats Bit positions (seat - 2) Mask (binary)

A	2–5	0,1,2,3	0b00001111
B	4–7	2,3,4,5	0b00111100
C	6–9	4,5,6,7	0b11110000

Step 4 — Check availability with AND. For a row's mask m , block X is fully free exactly when $m \& X == 0$. Since Block A and Block C never share bits, if both are free, that row scores 2. Otherwise, check B, and score at most 1 for the row.

Step 5 — Sum it up. Every row not in the dictionary contributes 2. Every row in the dictionary contributes 0, 1, or 2 depending on which blocks survive the AND check.

This collapses the ten-seat scan into a handful of integer operations per reserved row.

Python Code

```
from collections import defaultdict
from typing import List

def max_number_of_families(n: int, reserved_seats: List[List[int]]) -> int:
    BLOCK_A = 0b00001111 # seats 2-5
    BLOCK_B = 0b00111100 # seats 4-7
    BLOCK_C = 0b11110000 # seats 6-9

    row_masks = defaultdict(int)

    for row, seat in reserved_seats:
        if 2 <= seat <= 9:
            row_masks[row] |= 1 << (seat - 2)

    # Every row without a reservation seats two full groups.
    total_groups = (n - len(row_masks)) * 2

    for mask in row_masks.values():
        if mask & BLOCK_A == 0 and mask & BLOCK_C == 0:
            total_groups += 2
        elif (mask & BLOCK_A == 0) or (mask & BLOCK_B == 0) or (mask & BLOCK_C == 0):
            total_groups += 1

    return total_groups
```

Code Walkthrough

- `BLOCK_A`, `BLOCK_B`, `BLOCK_C` are constant bitmasks representing which of the 8 tracked seats (2 through 9) each block occupies. Bit 0 corresponds to seat 2, bit 7 to seat 9.
- `row_masks` is a `defaultdict(int)` so any row we touch starts at mask 0 (fully free) and we never need an explicit "row exists" check.
- The main loop over `reserved_seats` filters out seats 1 and 10 with the `2 <= seat <= 9` guard, then sets the corresponding bit with `|= 1 << (seat - 2)`.
- `total_groups` is initialized using every row that *never* appears in `row_masks` — those are fully empty and worth 2 groups each, computed in $O(1)$ instead of iterated.

- The final loop only runs over rows that actually have reservations. `mask & BLOCK_X == 0` tests whether none of that block's seats are set in the reservation mask — i.e., the block is fully open.
- The `elif` only fires if the A-and-C combo failed, and checks each block independently since at most one can be free at that point (if two of A, B, C were free without A-and-C both being free, that would only happen with B and one of A/C, and B overlaps both, so a row with B and A free would already have hit the first branch's logic if C is also free — the `elif` safely picks up the single-block case).

Dry Run

Reservations: `[(1,2), (1,3), (1,8), (2,6), (3,1), (3,10)]`, `n = 3`.

- Row 1: seat 2 → bit 0, seat 3 → bit 1, seat 8 → bit 6. Mask = `0b01000011`.
- Row 2: seat 6 → bit 4. Mask = `0b00010000`.
- Row 3: seats 1 and 10 are skipped (outside 2–9). Row 3 never enters `row_masks`.

`row_masks` has 2 entries (rows 1 and 2), so rows without reservations = `3 - 2 = 1` → contributes `1 * 2 = 2`.

Row 1, mask `0b01000011` : - `mask & BLOCK_A = 0b01000011 & 0b00001111 = 0b00000011 ≠ 0` → A blocked. - `mask & BLOCK_B = 0b01000011 & 0b00111100 = 0` → B free! Row 1 contributes 1.

Row 2, mask `0b00010000` : - `mask & BLOCK_A = 0` → A free. - `mask & BLOCK_C = 0b00010000 & 0b11110000 = 0b00010000 ≠ 0` → C blocked. - A-and-C combo fails. `elif` : A is free → contributes 1.

Total: `2 + 1 + 1 = 4`, matching the expected result.

Complexity Analysis

Time: $O(k)$ where `k` is the number of reservations. We touch each reservation exactly once to build masks, then iterate the dictionary once, whose size is bounded by `k`. This is correct because rows without reservations are never visited — they're accounted for with a single arithmetic subtraction.

Space: $O(k)$ for the dictionary, since only rows with at least one reservation get an entry. In the worst case (every row has a reservation), this is $O(n)$, but it never exceeds the number of distinct rows touched.

Alternative Solutions

A boolean-array-per-row approach (the brute force above) has the same asymptotic complexity but worse constants — array allocation and per-seat iteration versus integer bit operations. Another alternative is precomputing all 256 possible 8-bit masks into a lookup table of group counts, trading a small amount of preprocessing for $O(1)$ lookups per row instead of a few AND operations. For this problem size, the bitmask-with-AND approach already runs fast enough that the lookup table is unnecessary complexity.

Edge Cases

- **No reservations at all:** every row scores 2, total is `2n`. The main loop never runs.
- **All reservations on seats 1 or 10:** they're filtered out, every affected row still counts as empty and scores 2.
- **Duplicate reservations for the same seat:** setting the same bit twice with `|=` is idempotent — no effect on correctness.
- **A row with every seat from 2–9 reserved:** mask is `0b11111111`, all three block checks fail, row contributes 0.
- **$n = 1$ with minimal reservations:** works the same as any other row; the dictionary might hold just one entry.

Common Mistakes

1. **Assuming a row caps out at one group** because B overlaps A and C — missing that A and C are independent of each other.
2. **Including seats 1 and 10 in the mask**, wasting bits and complicating the block constants for seats that can never affect a block.
3. **Iterating all n rows explicitly** instead of computing empty-row contributions with a single multiplication.
4. **Off-by-one errors in bit positions** when mapping seat number to bit index — seat 2 must map to bit 0, not bit 2.
5. **Checking blocks with equality instead of AND-with-zero**, e.g., comparing against a fixed "free" pattern instead of testing `mask & BLOCK == 0`, which breaks if seats outside the block happen to be reserved.

Interview Questions

- How would this change if blocks weren't fixed and any four consecutive seats counted?
- How would you extend this to handle concurrent bookings safely?

- What if row length changed from 10 seats to an arbitrary number, breaking the fixed 8-bit mask?
- Can you solve this without a dictionary, using a different data structure, and what are the tradeoffs?
- How would you test this function, including adversarial inputs?

Similar Problems

- **Seat Reservation Manager (LeetCode 1845)** — different structure but same theme of tracking seat availability efficiently.
- **Bitwise AND of Numbers Range (LeetCode 201)** — good practice for bitmask reasoning and AND semantics.
- **Number of Provinces / Islands problems** — different domain, but similar "only process what's marked, skip the rest" optimization pattern.
- **Design Parking System (LeetCode 1603)** — another counting-with-constraints problem that rewards a compact state representation.

Key Takeaways

The core lesson isn't bitmasking itself — it's recognizing when a problem's search space is smaller than it first appears. Once you see there are only three fixed blocks per row, and that two of those three never conflict, the rest of the solution falls out naturally. Bitmasking is just the cleanest way to express that reduced search space in code.

Watch the Video

For the full walkthrough, including the timing-gap story about real double-bookings and a live run against LeetCode's test cases, watch the video here: {LINK}

About the Series

This breakdown is part of Fun with Learning Technology, a series where we take LeetCode Daily Challenge problems and dig past the accepted solution into the reasoning that gets you there — the same reasoning that shows up in real system design when these patterns extend past a single interview question.

Call To Action

If this helped the block trick click, leave a like on the video and subscribe for more daily LeetCode breakdowns. Subscribe to the Substack for the written version of every episode, drop a comment with the problem you want covered next, and share this with someone prepping for interviews.

Quick Review

Cinema Seat Allocation smells like which pattern? Trigger?

Bitmask row scan. Trigger: fixed-width rows, need contiguous free blocks of size k per row.

Time complexity for scanning all rows for valid seat blocks?

$O(n)$ where n = number of reservations/rows touched, since each row has fixed width (10 seats).

Space complexity of the row-bitmask approach?

$O(n)$ for the hashmap of row $\rightarrow$ occupied-seat-bitmask, not $O(\text{rows} \times \text{seats})$.

Trickiest bug in this problem?

Checking only pre-defined 4-seat blocks misses valid group-of-4 seats spanning two adjacent fixed blocks (e.g. seats 4-7).

Why must reservations be grouped/locked per row before scanning?

A single global lock lets concurrent bookings race on different rows; the real fix is per-row state, not one broad lock.

Follow-up: how to extend for arbitrary group size k , not just 4?

Generalize the bitmask check to a sliding window of width k across the row, not just the 3 fixed blocks.